



IT0049 (WEB SYSTEM TECHNOLOGIES)

TECHNICAL FORMATIVE ASSESSMENT

1

FROM ZERO TO FOUR PAGES: YOUR FIRST CODEIGNITER APPLICATION

Student Name / Group Name:	Pardo, Miguel
Section:	TC37
Professor:	Prof. Joseph Calleja

PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Analyze a complex problem and identify and define the computing requirements appropriate to its solution. [PO: B]

COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Understand the advanced principles of web development using web frameworks and MVC architecture. [CLO: 1]

INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Install CodeIgniter using Composer and configure the application's base URL.
- Create routes for a multi-page website.
- Build controllers that respond to those routes.
- Create views and pass data to them.
- Use a static PHP array as a temporary data source for a listing page.

BACKGROUND INFORMATION

Before a web application can talk to a database, it needs a clear structure for turning a URL into a page. CodeIgniter organizes this using the Model-View-Controller (MVC) pattern: routes decide which controller method runs for a given URL, the controller decides what to do, and the view renders the HTML the browser receives. This activity focuses on the routing, controller, and view layers only — the data layer comes in the next module.

A static PHP array is a simple way to represent a list of records before a real database is introduced. It behaves like a temporary, in-memory table: you can loop over it in a view exactly the way you will later loop over database results, which is why this activity uses it as a stand-in.

GRADING SYSTEM / RUBRIC

CRITERIA	Description
Functionality & Requirements Completeness (40 points max)	All required pages and features are present and work as specified in the activity instructions.
Code Structure & Organization (25 points max)	Controllers, models, and views are separated correctly and follow the framework's conventions; code is readable and sensibly named.
Data Handling with Static Arrays (20 points max)	The static PHP array is structured sensibly, used consistently as the page's temporary data source, and looped through correctly in the view.
Documentation & Submission Quality (15 points max)	The GitHub repository is organized and complete, the README explains how to set up and run the project, and the hosted link is live and matches the submitted code.

LABORATORY ACTIVITY

You will build the first version of a basic Point-of-Sale (POS) system: a four-page CodeIgniter website. The Customer Accounts and User Accounts pages will use a static PHP array as a temporary data source — no database is involved yet.

Required pages: a landing page (/), an about page (/about), a Customer Accounts page (/customers) listing customer records (full name, email, phone) from a static array, and a User Accounts page (/users) listing user/staff records (username, full name, role) from a static array.

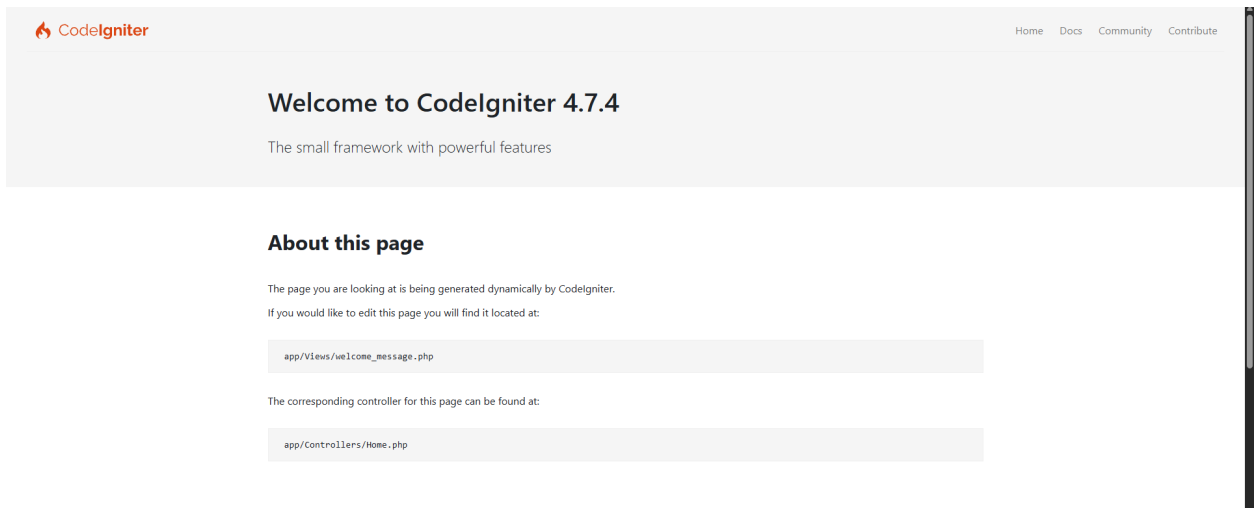
1. Install CodeIgniter 4 via Composer and confirm the base project runs in your browser.

```
PS C:\Users\mmpardo\Downloads\TFA1--Web-System-Technologies\TFA1> php spark serve

CodeIgniter v4.7.4 Command Line Tool - Server Time: 2026-09-17 11:27:41 UTC+00:00

CodeIgniter development server started on http://localhost:8080
Press Control-C to stop.
[Thu Sep 17 19:27:41 2026] PHP 8.2.12 Development Server (http://localhost:8080) started
```

2. Configure your app's base URL in .env and clean up the default welcome routes.



3. Create a Pages controller with methods for the landing and about pages, and register their routes.

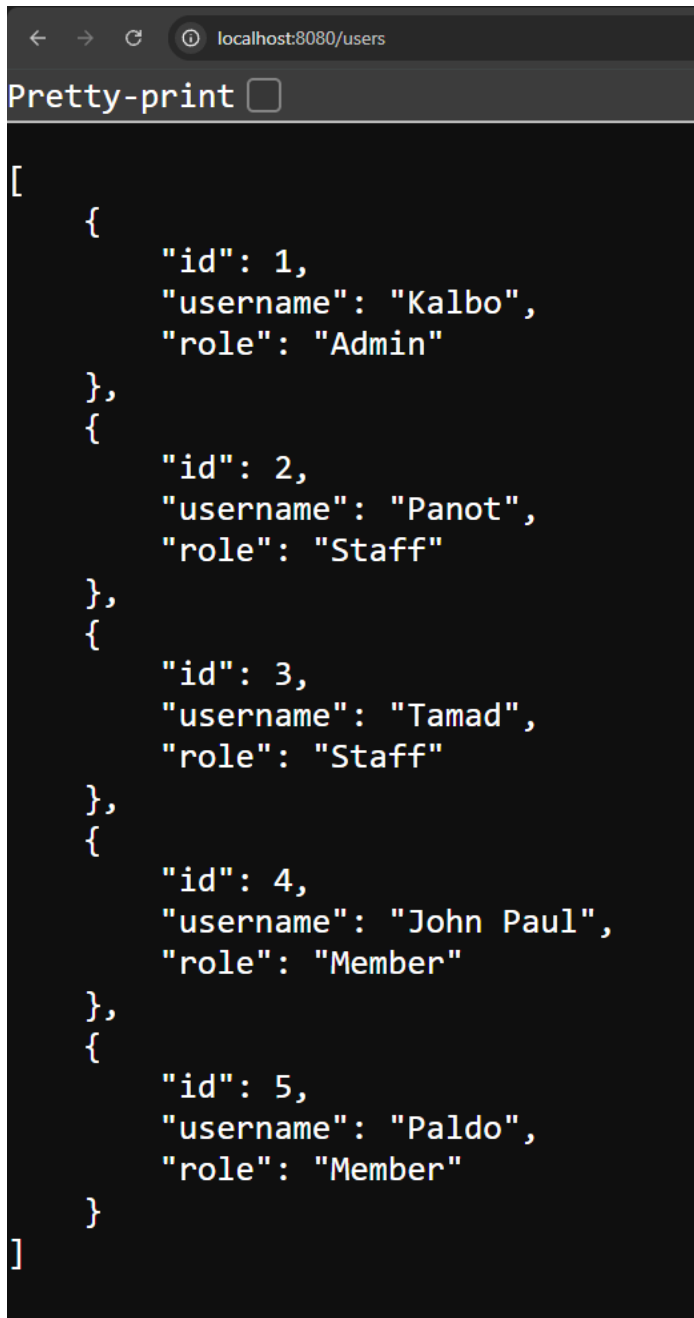


ABOUT PAGE

LANDING PAGE

4. Create a Customers controller and a Users controller. In each, define a static PHP array of at least 5 sample records as the method's temporary data source.

```
localhost:8080/customers
Pretty-print ☐
[
  {
    "id": 1,
    "name": "Freddy Fazbear",
    "email": "freddyfazbear@example.com"
  },
  {
    "id": 2,
    "name": "Mickey Mouse",
    "email": "mickeymouse@example.com"
  },
  {
    "id": 3,
    "name": "Charlie Kirk",
    "email": "charliekirk@example.com"
  },
  {
    "id": 4,
    "name": "Dinorobong Tubo",
    "email": "dinorobongtubo@example.com"
  },
  {
    "id": 5,
    "name": "Kween Yasmin",
    "email": "kweenyasmin@example.com"
  }
]
```

A screenshot of a web browser window. The address bar shows 'localhost:8080/users'. Below the address bar is a dark grey header with the text 'Pretty-print' followed by a small square icon. The main content area has a black background with white text displaying a JSON array of five user objects. The objects are: 1. {id: 1, username: 'Kalbo', role: 'Admin'}, 2. {id: 2, username: 'Panot', role: 'Staff'}, 3. {id: 3, username: 'Tamad', role: 'Staff'}, 4. {id: 4, username: 'John Paul', role: 'Member'}, 5. {id: 5, username: 'Paldo', role: 'Member'}.

```
[
  {
    "id": 1,
    "username": "Kalbo",
    "role": "Admin"
  },
  {
    "id": 2,
    "username": "Panot",
    "role": "Staff"
  },
  {
    "id": 3,
    "username": "Tamad",
    "role": "Staff"
  },
  {
    "id": 4,
    "username": "John Paul",
    "role": "Member"
  },
  {
    "id": 5,
    "username": "Paldo",
    "role": "Member"
  }
]
```

5. Build views for all four pages. In the Customer Accounts and User Accounts views, loop through the static array data with a foreach to display each record.

Customer Accounts

ID	Name	Email
1	Freddy Fazbear	freddyfazbear@example.com
2	Mickey Mouse	mickeymouse@example.com
3	Charlie Kirk	charliekirk@example.com
4	Dinorobong Tubo	dinorobongtubo@example.com
5	Kween Yasmin	kweenyasmin@example.com

User Accounts

ID	Username	Role
1	Kalbo	Admin
2	Panot	Staff
3	Tamad	Staff
4	John Paul	Member
5	Paldo	Member

6. Add simple navigation links between all four pages and confirm every route works.

[Home](#) [About](#) [Customer Accounts](#) [User Accounts](#)

HELLO

This is the landing page.

[Home](#) [About](#) [Customer Accounts](#) [User Accounts](#)

ABOUT

This is the about page.

S

[Home](#) [About](#) [Customer Accounts](#) [User Accounts](#)

User Accounts

ID	Username	Role
1	Paul John	Admin
2	John Doe	Staff
3	Horsey	Staff
4	John Paul	Member
5	Bingo Plus	Member

[Home](#) [About](#) [Customer Accounts](#) [User Accounts](#)

Customer Accounts

ID	Name	Email
1	Freddy Fazbear	freddyfazbear@example.com
2	Mickey Mouse	mickeymouse@example.com
3	Charlie Kirk	charliekirk@example.com
4	Dinorobong Tubo	dinorobongtubo@example.com
5	Kween Yasmin	kweenyasmin@example.com

Submission Requirements:

- A link to a GitHub repository containing your raw project files, including your database export.
- A link to your hosted, working version of the application.

REFERENCES

- Manfield, A. (2021). Pro CodeIgniter: Learn how to create professional web-applications with PHP.
- CodeIgniter User Guide. Routing.
<https://codeigniter4.github.io/userguide/incoming/routing.html>
- CodeIgniter User Guide. Controllers.
<https://codeigniter4.github.io/userguide/incoming/controllers.html>